

PROYECTO COLABORATIVO · CODELAB

Sistema de Gestión de Inventario con Consultas SQL Dinámicas

DSL / Motor de consultas de datos relacionales

Embedded DSL: SQL en JavaScript

DOCENTE

Ing. Jorge Medianero Acosta

SEMESTRE

2026-I

TIPO DE PROYECTO

Proyecto colaborativo — 4 integrantes

FECHA

Lima, Perú — Junio 2026

INTEGRANTES DEL EQUIPO

- Dino Paul Roel Suazo Zanabria
- Farid Paolo Zamudio Sanchez
- Luis Angel Vargas Ponce
- Yazid Elio Fernandez Dueñas

Nombre del Caso de Uso	Realizar Consulta SQL Dinámica sobre el Inventario
ID	UC-001
Actor Principal	Usuario del sistema (Administrador / Analista)

Precondiciones

- ▶ El usuario debe estar autenticado en el sistema con un rol asignado (admin o viewer).
- ▶ La base de datos de inventario debe estar inicializada con al menos una tabla de productos.

Postcondiciones

- ▶ El sistema ejecuta la consulta SQL generada y muestra los resultados en pantalla.
- ▶ Los resultados pueden ser visualizados en tabla o gráfico, y exportados en el formato deseado.

Flujo Principal de Eventos

- 1 El usuario accede al sistema e inicia sesión con sus credenciales (username + password).
- 2 El sistema autentica al usuario y asigna su rol correspondiente (admin o viewer).
- 3 El usuario configura la consulta: selecciona columnas, define filtros, orden y límite de resultados.
- 4 El sistema genera automáticamente la consulta SQL a partir de los parámetros seleccionados.
- 5 El usuario revisa el SQL generado en el área de previsualización en tiempo real.
- 6 El usuario solicita validar la consulta; el sistema verifica su sintaxis y permisos de rol.
- 7 El usuario ejecuta la consulta; el sistema la procesa contra la base de datos SQLite.
- 8 El sistema retorna los resultados y los muestra en tabla interactiva y gráficos automáticos.
- 9 El usuario exporta el reporte en CSV o PDF si lo requiere.

Flujos Alternativos

- ▶ Si la consulta generada es inválida, el sistema muestra un mensaje de error y no ejecuta la operación.
- ▶ Si el usuario tiene rol viewer e intenta una operación de escritura, el sistema deniega la acción con HTTP 403.
- ▶ Si los resultados superan 1000 filas, el sistema los trunca automáticamente.

1.b

User Story (Historia de Usuario)

Título

Consulta visual de inventario mediante SQL dinámico

AS A (Como)

Administrador de inventario

I WANT (Quiero)

Construir consultas SQL de forma visual usando un formulario interactivo, sin necesidad de escribir SQL manualmente

SO THAT (Para que)

Pueda obtener reportes personalizados del inventario de manera rápida y sin conocimientos avanzados de bases de datos

Criterios de Aceptación

El sistema debe generar una consulta SQL válida a partir de los filtros seleccionados por el usuario en el formulario.

Antes de ejecutar, el sistema debe validar la consulta y mostrar un mensaje de error descriptivo si es inválida.

Los resultados deben mostrarse tanto en tabla como en al menos un tipo de gráfico (barras o torta).

El usuario con rol **viewer** solo puede ejecutar consultas SELECT; el **admin** puede ejecutar INSERT, UPDATE y DELETE.

El sistema debe permitir exportar los resultados en CSV y en PDF.

2.a Arquitectura WEB

El proyecto sigue una arquitectura **cliente-servidor de tres capas**, donde cada capa tiene responsabilidades bien definidas y se comunican mediante una API REST:

Capa	Responsabilidad	Tecnología
Frontend	Interfaz de usuario, formulario de consultas, gráficos, exportación	React + Vite + CSS
Backend	API REST, lógica de negocio, validación SQL, control de acceso por roles	Node.js + Express
Base de Datos	Almacenamiento persistente del inventario de productos	SQLite (better-sqlite3)

Flujo de comunicación

- 1 El usuario configura los filtros en el formulario del Frontend (React).
- 2 El Frontend genera el SQL y lo envía al Backend vía HTTP POST a `/api/query/execute`.
- 3 El Backend valida la consulta (sintaxis + permisos de rol) y la ejecuta contra SQLite.
- 4 El resultado se retorna como JSON y el Frontend lo renderiza en tabla y gráficos.

2.b DSL Utilizado

El DSL utilizado en este proyecto es **SQL (Structured Query Language)**. SQL es formalmente un DSL porque cumple con las tres características clave:

- ▶ **Dominio Específico:** diseñado exclusivamente para consultar y manipular bases de datos relacionales.
- ▶ **Capacidad de Expresión:** permite definir selecciones, filtros, agrupaciones y transformaciones de datos.
- ▶ **Abstracción:** oculta la complejidad del almacenamiento físico y la recuperación de datos.

Implementación: Embedded DSL

SQL se encuentra embebido dentro de JavaScript (Node.js). El sistema genera sentencias SQL dinámicamente a partir de los parámetros del formulario. El usuario nunca escribe SQL directamente — la interfaz lo construye automáticamente. Esto constituye un *Embedded DSL*, donde el lenguaje huésped es JavaScript y el DSL es SQL.

Capa	Tecnología	Propósito
Frontend	React 18 + Vite	Framework UI declarativo con renderizado reactivo y build optimizado
Frontend	CSS (dark theme)	Estilos personalizados con variables CSS para diseño oscuro profesional
Backend	Node.js v18	Entorno de ejecución JavaScript del lado del servidor
Backend	Express 4	Framework web minimalista para construir la API REST
Base de Datos	SQLite + better-sqlite3	Motor de base de datos embebido, sincrónico y sin configuración adicional
Seguridad	JWT (jsonwebtoken)	Autenticación stateless con tokens firmados y expiración de 24 horas
Seguridad	bcryptjs	Hash seguro de contraseñas con sal para almacenamiento en base de datos
Visualización	Recharts 2	Librería de gráficos React: BarChart y PieChart responsivos
Exportación	jsPDF + autotable	Generación de reportes PDF en el cliente con tablas formateadas
Exportación	PapaParse 5	Serialización y descarga de datos en formato CSV estándar

4.a Backend — API REST

El backend expone los siguientes endpoints REST agrupados en dos rutas principales:

Endpoint	Método	Descripción
/api/auth/login	POST	Autentica usuario y retorna JWT firmado
/api/auth/me	GET	Retorna info del usuario autenticado (requiere token)
/api/query/schema	GET	Retorna columnas y tipos de la tabla productos
/api/query/build	POST	Recibe descriptor JSON y genera el SQL correspondiente
/api/query/validate	POST	Valida la sintaxis SQL sin ejecutarla
/api/query/execute	POST	Ejecuta la consulta y retorna resultados (máx. 1000 filas)

4.b Frontend — Query Builder

El componente **QueryBuilder** es el núcleo de la interfaz. Permite al usuario:

- ▶ Seleccionar columnas mediante checkboxes individuales o toggle global.
- ▶ Agregar múltiples filtros dinámicos con columna, operador (=, !=, >, <, LIKE, IS NULL, etc.) y valor.
- ▶ Definir ordenamiento (columna + dirección ASC/DESC) y límite de resultados (máx. 1000).
- ▶ Previsualizar el SQL generado en tiempo real a medida que edita los parámetros del formulario.
- ▶ Validar y ejecutar la consulta con botones dedicados que muestran estados de carga.

4.c Sistema de Roles y Seguridad

El control de acceso se implementa mediante middleware JWT en el backend. Cada petición autenticada es validada antes de procesarse:

- ▶ **Rol admin:** puede ejecutar SELECT, INSERT, UPDATE y DELETE sobre los datos.
- ▶ **Rol viewer:** solo puede ejecutar consultas SELECT. Cualquier otra operación es rechazada con HTTP 403.
- ▶ **Ambos roles:** las operaciones destructivas (DROP, TRUNCATE, ALTER) están siempre prohibidas.

◆ **Implementación exitosa del DSL**

Se implementó SQL como DSL embebido en JavaScript, demostrando en la práctica cómo un lenguaje de dominio específico puede integrarse en una aplicación web moderna para automatizar la generación de consultas relacionales.

◆ **Accesibilidad para usuarios no técnicos**

El Query Builder visual elimina la barrera de conocimiento SQL, permitiendo a cualquier usuario construir consultas complejas a través de un formulario intuitivo sin escribir una sola línea de código.

◆ **Arquitectura escalable**

La separación en tres capas independientes (React / Express / SQLite) facilita el mantenimiento y permite migrar la base de datos a PostgreSQL en producción sin modificar la lógica de negocio del backend.

◆ **Seguridad robusta mediante roles**

La combinación de JWT y bcrypt con control de roles garantiza que cada usuario solo pueda ejecutar las operaciones que su nivel de acceso permite, cumpliendo con principios de mínimo privilegio.